PROPUESTA DE PROYECTO FINAL

Proyecto: Protocolo Cero - Juego de decisiones y programación

Integrantes:

Fidel Sebastian Aguilar Morejón
Boris Ignacio Calcina Barriga

1. DESCRIPCIÓN DEL DOMINIO DEL PROYECTO

Protocolo Cero es un videojuego educativo de aventura narrativa y resolución de problemas mediante programación. El jugador interpreta a un ingeniero que despierta en una estación científica con fallos en sus sistemas de energía, seguridad y ventilación. Para avanzar debe tomar decisiones y escribir instrucciones que controlen distintos dispositivos de la estación.

El problema que busca abordar es la dificultad de los estudiantes principiantes para relacionar conceptos de programación, como condiciones, secuencias e instrucciones, con sus efectos sobre un sistema. La propuesta permite experimentar con estos conceptos dentro de situaciones simuladas, en las que cada decisión o programa produce consecuencias visibles.

El sistema está dirigido principalmente a estudiantes que se inician en programación. Administra misiones, personajes, decisiones, recursos disponibles, dispositivos, instrucciones escritas por el jugador y estados del juego. Se plantea una primera versión con tres misiones conectadas y diferentes desenlaces según las decisiones tomadas.

2. MODELO DE DATOS

El modelo se implementará en Scala y priorizará datos inmutables. Sus entidades principales serán:

• Jugador: identifica al personaje y almacena sus características básicas, como salud y nombre.

• EstadoJuego: representa la situación actual, incluyendo misión, energía, temperatura, seguridad, dispositivos y decisiones anteriores.

• Mision: contiene la descripción, los objetivos, las opciones disponibles y las condiciones para avanzar.

• Decision: define una elección del jugador, sus requisitos y los cambios que produce en el estado.

• Dispositivo: representa puertas, ventiladores, generadores u otros sistemas, con su tipo, estado y consumo de recursos.

• Instruccion y Programa: representan las órdenes de un lenguaje sencillo, como activar un dispositivo o ejecutar una acción bajo una condición. Un programa contiene una lista de instrucciones.

• ResultadoEjecucion[A]: representa de forma genérica el resultado de una operación, con un valor válido o la información de un error.

Se utilizarán case class para las entidades, sealed trait para tipos de instrucciones y acciones, List para colecciones y pattern matching para interpretar órdenes. Las operaciones map, filter y foldLeft permitirán procesar dispositivos, verificar condiciones y aplicar instrucciones secuencialmente. Cada operación devolverá un nuevo EstadoJuego sin modificar el anterior.

3. LISTA DE FUNCIONALIDADES

1. Iniciar una partida y presentar la historia y el objetivo de cada misión.

2. Mostrar alternativas de decisión que modifican recursos, acontecimientos y misiones posteriores.

3. Consultar el estado de la estación: energía, temperatura, seguridad y dispositivos activos.

4. Escribir programas en una terminal con un lenguaje simplificado que admita órdenes como ACTIVAR, DESACTIVAR y condiciones del tipo SI ... ENTONCES ... FIN.

5. Analizar y validar las instrucciones, mostrando mensajes cuando existan errores de escritura o de uso de comandos.

6. Interpretar programas válidos y aplicar sus efectos sobre una copia lógica del estado, sin modificar directamente los datos anteriores.

7. Resolver situaciones con más de una estrategia, combinando decisiones y programación.

8. Mostrar los resultados de las acciones y ofrecer distintos desenlaces de acuerdo con el progreso y las decisiones realizadas.

4. INTERFAZ GRÁFICA ASOCIADA

Se propone una interfaz gráfica sencilla de escritorio, implementada en Scala con componentes de Java Swing, separada de la lógica del juego. Estará organizada en las siguientes pantallas:

• Menú principal: título del juego y opciones para iniciar o salir.

• Pantalla de misión: descripción de la situación, opciones de decisión y panel con los recursos disponibles.

• Terminal de programación: área de texto para escribir instrucciones, botón para ejecutarlas y zona donde se muestran errores y resultados.

• Pantalla de resultados: consecuencias de la decisión o programa, cambios de estado y acceso a la siguiente misión.

La interfaz priorizará la claridad de los textos y controles. El procesamiento de instrucciones, las reglas y la evolución de la partida se implementarán como funciones separadas de la presentación gráfica.